

课程笔记

CME 295 第 4 讲: LLM Training

Codex 基于 Stanford CME 295 第 4 讲视频、字幕、讲稿与 slides 整理

2026 年 4 月 13 日



视频作者/频道: Stanford CME 295

发布日期: 2025 年 10 月 17 日课程录播

视频时长: 1:47:27

视 频 链 接: https://www.youtube.com/watch?v=VlA_jt_3Qc4&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=4

目录

1 训练范式：从单任务模型到“预训练 + 调优”	2
1.1 为什么 LLM 训练先学通用能力，再学任务行为	2
1.2 两阶段范式改变了数据、成本和模型复用方式	2
1.3 本章小结	3
2 预训练：让模型先学会语言与代码	3
2.1 预训练的目标是最大化下一个 token 的可预测性	3
2.2 数据混合比单一数据更关键	3
2.3 规模、样本效率与 Chinchilla 法则	3
2.4 预训练为什么又贵又难	4
2.5 本章小结	4
3 训练优化：显存、并行与算子层面的加速	4
3.1 训练瓶颈首先是显存，而不是纸面算力	4
3.2 数据并行和 ZeRO：把冗余状态拆开存	4
3.3 模型并行与训练切分维度	5
3.4 FlashAttention：训练加速不只是分卡，也要减少 HBM 访存	5
3.5 混合精度：用更低精度换更高吞吐和更低显存	6
3.6 本章小结	6
4 监督微调：把“会续写”变成“会配合”	6
4.1 SFT 的角色是让模型学会特定行为	6
4.2 Instruction tuning 的数据比预训练小得多，但质量要求更高	7
4.3 SFT 改变的是行为，不是全部能力边界	7
4.4 SFT 评估仍然很难	7
4.5 本章小结	7
5 参数高效微调：LoRA、QLoRA 与低成本适配	7
5.1 LoRA：不改满矩阵，而是学习一个低秩增量	7
5.2 LoRA 的价值不只是省显存，还在于可插拔	8
5.3 LoRA 应该加在哪里，以及它的训练动态有什么特殊性	8
5.4 QLoRA：把冻结权重量化，再保留全精度 LoRA 分支	9
5.5 QLoRA 说明了“显存预算”本身就是建模约束	10
5.6 本章小结	10
6 总结与延伸	10

1 训练范式：从单任务模型到“预训练 + 调优”

1.1 为什么 LLM 训练先学通用能力，再学任务行为

这一讲先把训练范式讲清楚。传统机器学习里，一个任务往往对应一个模型：垃圾邮件检测训练一个模型，情感分类再训练一个模型，翻译还要单独再来一次。CME 295 强调，语言任务之间并不是完全割裂的，它们共享对语言结构、知识组织、常识与代码模式的理解，所以更合理的做法不是每次从零开始，而是先训练出一个具有广泛语言能力的基础模型，再把它适配到下游任务。

课程把这一过程拆成两段：第一段是 **pretraining**，目标是让模型学会语言与代码的统计结构；第二段是更广义的 **tuning**，目标是让模型在具体使用场景中表现得更像人类需要的系统。这里最重要的思想不是“模型变大了”，而是“训练目标被分层了”：先解决知识与表示能力，再解决任务行为与交互风格。

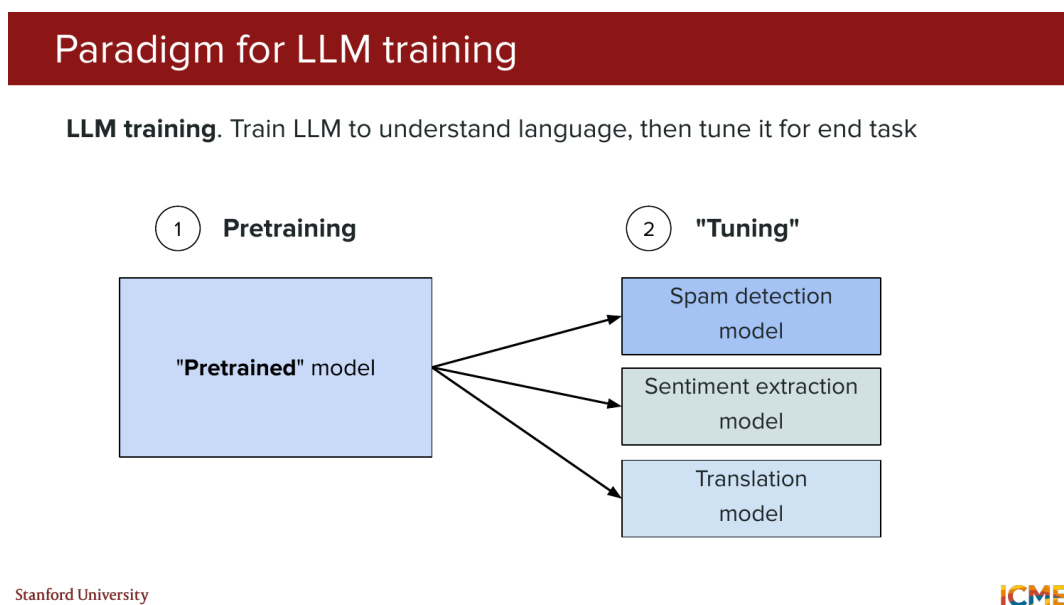


图 1: 课程用两阶段示意图说明 LLM 的主流训练范式：先预训练一个通用模型，再把它适配到具体任务。

1.2 两阶段范式改变了数据、成本和模型复用方式

这个范式带来三层变化。第一，**数据来源改变了**：预训练阶段使用的是规模极大的开放文本、网页和代码混合数据；微调阶段则使用规模更小但质量更高、标签更明确的数据。第二，**成本结构改变了**：真正昂贵的是预训练，它通常消耗大规模 GPU 集群、长时间训练和巨额电力成本；调优虽然仍然昂贵，但一般比预训练小得多。第三，**模型复用方式改变了**：一个预训练模型可以被适配成聊天助手、分类器、翻译器、代码助手，而不必为每个任务重建完整知识底座。

本讲的总纲

Lecture 4 的逻辑是：先解释为什么 LLM 训练要分阶段，再讨论大模型为什么难训练，接着讲硬件和算法层面的训练优化，最后进入 SFT、LoRA 与 QLoRA 这类面向下游适配的方案。

1.3 本章小结

本节的核心不是某个具体公式，而是训练思想的切换。LLM 不再是“一个任务一个模型”，而是“一个基础模型，多种任务适配”。后面所有关于显存、并行、微调和量化的讨论，都是围绕这个分层训练范式展开的。

2 预训练：让模型先学会语言与代码

2.1 预训练的目标是最大化下一个 token 的可预测性

课程把预训练定义得很朴素：给模型大量文本和代码，让它反复做 **next-token prediction**。如果输入序列是 $x_1, x_2, \dots, x_T$ ，那么训练目标可以写成

$$\max_{\theta} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}).$$

符号说明如下：

- θ ：模型参数；
- x_t ：序列中第 t 个 token；
- $x_{<t}$ ：第 t 个 token 之前的上下文；
- $p_{\theta}(x_t | x_{<t})$ ：在参数为 θ 的模型下，第 t 个 token 的条件概率。

这里的含义不是让模型“背答案”，而是让它在海量上下文中反复学习语言模式、语法结构、常识关联、代码模板、跨语言映射和领域术语之间的统计关系。课程特别强调，现代 LLM 的预训练语料不只是英文网页，还包含多语言文本和多种编程语言，因此“会写代码”和“会做跨语言推断”并不是额外模块，而是预训练目标在不同分布上的自然外推。

2.2 数据混合比单一数据更关键

Slides 明确给出了预训练数据的几个主要来源：网页抓取数据、百科类知识、代码语料，以及更一般的多语言内容。这里值得注意的是，课程并没有把重点放在“某个单一黄金数据集”，而是放在 **data mixture** 上。原因很直接：如果希望模型既能理解自然语言，又能生成代码，还能在不同语言之间迁移能力，那么训练数据本身就必须覆盖这些分布。

从字幕里的口述可以看出，老师想传达的不是“全部互联网都值得学”，而是“预训练在工程上必须面对大规模、噪声、高异质性数据”。这也是后面训练成本极高、数据治理困难、知识截止与版权问题频繁出现的背景。

2.3 规模、样本效率与 Chinchilla 法则

Slides 用数页内容回顾了 scaling、sample efficiency 和 Chinchilla law。课程的重点不是推导这些论文里的所有细节，而是给学生一个工程直觉：**更大的模型不一定总是更好，关键在于模型参数量与训练 token 量之间是否匹配。**

Kaplan 等人的 scaling laws 告诉我们，模型性能会随着模型规模、数据规模与训练计算量的增加呈现可预测改善；Hoffmann 等人的 Chinchilla 工作进一步指出，在给定训练算力预算下，很多模型其实是“参数太多、token 太少”，因此没有达到计算最优。也就是说，预训练不是盲目堆参数，而是要考虑“模型是否被足够多的数据喂饱”。

2.4 预训练为什么又贵又难

Lecture 4 很早就列出了预训练的几个痛点：

- 成本极高，往往是百万美元量级甚至更高；
- 训练周期长，实验反馈慢；
- 知识天然存在 cutoff，训练结束后世界还在变化；
- 已学到的知识难以局部编辑；
- 数据来源复杂，会触及环境成本与版权争议。

这些挑战说明，预训练并不是“先花钱做完再说”的前置步骤，而是整个 LLM 生命周期中最难逆转、最难局部修补的一步。预训练阶段学到了什么、漏掉了什么、用什么分布学到的，会持续影响后续所有微调与对齐步骤。

关于知识截止的现实含义

预训练模型的知识不是在线更新的数据库，而是训练结束时被压进参数里的静态近似。因此，当世界事实发生变化时，模型可能仍然表现得很自信，但答案已经过时。

2.5 本章小结

预训练的本质是“在海量异质数据上反复做 next-token prediction”，让模型形成通用语言与代码能力。规模法则告诉我们为什么大模型有效，Chinchilla 法则提醒我们不能只堆参数，而成本、知识截止和数据治理问题则解释了为什么预训练是整条链路里最难的一步。

3 训练优化：显存、并行与算子层面的加速

3.1 训练瓶颈首先是显存，而不是纸面算力

进入训练优化部分后，课程先把训练过程中真正占显存的对象列了出来：模型参数、前向激活、反向梯度和优化器状态。对于 Adam 一类优化器来说，参数本身之外还要维护一阶、二阶动量，因此显存压力远大于“只存一份权重”这件事。课程强调，当模型达到数十亿乃至数百亿参数时，单卡显存很快成为首要瓶颈。

3.2 数据并行和 ZeRO：把冗余状态拆开存

最直观的并行方式是数据并行：把 batch 切给多张 GPU，每张卡复制同一份模型，各自做前后向，再聚合梯度。它的优点是逻辑简单，缺点是冗余太多，特别是参数、梯度和优化器状态在每张卡上都来一份。

ZeRO (Zero Redundancy Optimization) 就是为了解决这种冗余。课程按阶段介绍了 ZeRO-1、ZeRO-2、ZeRO-3：

- ZeRO-1 共享优化器状态；
- ZeRO-2 再进一步共享梯度；

- ZeRO-3 连参数本身也做分片。

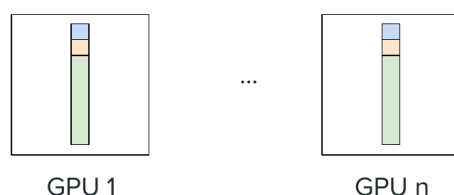
阶段越往后，单卡省下来的显存越多，但通信和实现复杂度也越高。

Data parallelism with ZeRO

Idea. Redundant information across devices

ZeRO = Zero Redundancy Optimization

Variations. ZeRO-3: Share **optimizer state** + **gradients** + **parameters**



Stanford University

"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models", Rajbhandari et al., 2019.

ICME

图 2: ZeRO-3 的核心思想是把优化器状态、梯度和参数都分布到多张 GPU 上，减少数据并行中的冗余副本。

3.3 模型并行与训练切分维度

除了数据并行，课程还列出了模型并行的多种切分方式：tensor parallelism、pipeline parallelism、sequence parallelism、context parallelism 以及专家并行。它们分别从参数维度、层级维度、序列维度或专家路由维度切开模型，让单张 GPU 只承担完整计算图中的一部分。

这里的教学重点建立“切分维度”的意识：当单卡装不下、单节点带宽不够、序列过长或模型包含稀疏专家结构时，工程师可以选择不同的并行维度，而不是只会加更多显卡做朴素复制。

3.4 FlashAttention：训练加速不只是分卡，也要减少 HBM 访存

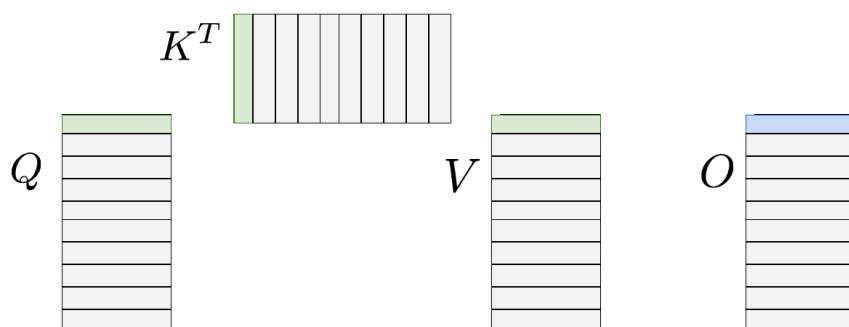
Lecture 4 选了 FlashAttention 作为算子层面优化的代表。课程先回顾了标准 self-attention 的执行路径：把 QK^T 写回 HBM，再从 HBM 读出来做 softmax，再写回概率矩阵，再读出来与 V 相乘。问题在于，HBM 大但慢，反复读写中间矩阵会让 attention 变成明显的 IO-bound 操作。

FlashAttention 的核心思路是：

- 用分块（tiling）策略，把更小的块放进片上 SRAM；
- 在 SRAM 内局部计算并维护 softmax 所需统计量；
- 避免显式物化完整的注意力矩阵与概率矩阵；
- 在反向传播中适度重计算，以换取更低显存占用和更少 IO。

Flash Attention in action

Load blocks of Q , K , V to **SRAM**, compute block of O and write the result to **HBM**



Stanford University "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", Dao et al., 2022.

ICME

图 3: FlashAttention 的示意图强调了它的关键动作：把 Q 、 K 、 V 分块搬到 SRAM，在片上直接算出输出块，再写回 HBM。

3.5 混合精度：用更低精度换更高吞吐和更低显存

课程后半段还回顾了浮点数表示和 mixed precision training。直觉上，数值精度越低，单个数占的字节越少，硬件每个周期可以处理的元素越多，于是训练速度和显存利用率都会改善。Slides 比较了 FP16、FP32、FP64 和 BF16 的指数位与尾数位差异，帮助学生理解为什么 BF16 在深度学习里尤其常见：它保留了较大的指数范围，同时把位宽压到 16 bit。

课程给出的实践建议是：前向和反向里尽量使用低精度张量以节省资源，但关键权重更新要小心数值稳定性。也就是说，混合精度训练并不是“全部改成低精度”这么简单，而是一个在速度、显存和数值稳定性之间做折中的系统设计。

3.6 本章小结

训练优化的三条主线很清晰：第一，显存压力来自参数、梯度、激活和优化器状态；第二，并行训练通过数据并行、模型并行和 ZeRO 等策略把状态和计算拆散；第三，FlashAttention 和混合精度说明，真正高效的训练还要深入到算子和数值表示层面减少 IO 与内存消耗。

4 监督微调：把“会续写”变成“会配合”

4.1 SFT 的角色是让模型学会特定行为

课程把 SFT (Supervised Fine-Tuning) 定义为：在预训练模型的基础上，用高质量输入输出对继续做 next-token prediction，从而把模型从“会自动补全的语言模型”转化为“愿意按照指令办事的助手”。这一步的重点不在于让模型知道更多世界知识，而在于让模型在给定提示格式下做出人希望看到的回应。

字幕里老师特别强调，预训练后的模型虽然已经懂语言和代码，但仍然更像一个“强大的续写器”，不一定知道如何成为一个有帮助、符合场景预期的系统。SFT 的任务就是教会模型回答问题、

遵循 instruction、给出结构化输出、使用特定语气和任务格式。

4.2 Instruction tuning 的数据比预训练小得多，但质量要求更高

Slides 给出的 instruction tuning 视角很清楚：输入通常是某个任务指令，输出是符合预期的助手回答。对应目标函数仍然是条件语言建模，只不过数据分布变成了“人类希望看到的互动”。课程也明确提到，这类数据既可以由人工撰写，也可以包含合成数据。

与预训练相比，SFT 有三个鲜明特征：

- 数据量小得多，往往是千到百万级样本，而不是万亿 token；
- 数据质量要求更高，因为模型会非常敏感地吸收这些行为模式；
- prompt 分布很重要，数据稍微失衡就可能让模型在某些场景下偏得很厉害。

4.3 SFT 改变的是行为，不是全部能力边界

课程用“能不能把泰迪熊放进洗衣机”这个例子展示了预训练模型和 instruction tuned 模型的区别。预训练模型可能会继续描述材质、结构或中性事实；instruction tuned 模型则更可能直接给出有帮助、可执行、贴合用户意图的建议。这说明 SFT 主要改变的是**行为接口**：同样的知识底座，在不同训练分布下会表现出完全不同的交互风格。

4.4 SFT 评估仍然很难

Slides 在 SFT 部分还列出了常见 benchmark，例如 MMLU、ARC-Challenge、GSM8K 和 HumanEval。它们分别覆盖通识知识、基础推理、数学推理与代码生成等维度。课程用这些 benchmark 传达一个事实：SFT 之后的模型是否“更好”，并不能由单一分数决定，因为模型会在不同维度上呈现不同强弱。

老师还用 Chatbot Arena 一类平台说明“真实用户感受”的重要性。它们能把“哪个模型更顺手、更讨喜”转成某种排序信号，但也存在曝光不均、容易被刷分、用户难以准确判断事实性等问题。也就是说，SFT 评估同时需要离线 benchmark 和在线偏好信号，单靠任一侧都不够。

4.5 本章小结

SFT 的本质不是给模型“再灌一次知识”，而是把预训练模型塑造成某种特定行为接口。它依赖高质量、小规模、分布设计良好的 instruction 数据，能够显著改善交互表现，但评估仍然是一个多维度、难以被单分数概括的问题。

5 参数高效微调：LoRA、QLoRA 与低成本适配

5.1 LoRA：不改满矩阵，而是学习一个低秩增量

Lecture 4 最后把重点放在参数高效微调（PEFT）上，其中 LoRA 是主角。它的核心思想是：冻结原始权重 W_0 ，只学习一个低秩增量 BA ，于是新的线性层写成

$$W = W_0 + BA.$$

符号说明如下：

- W : 微调后的有效权重;
- W_0 : 冻结的原始预训练权重;
- B, A : LoRA 额外学习的低秩矩阵;
- BA : 对原始权重的低秩增量修正。

其中 $B \in \mathbb{R}^{d \times r}$ 、 $A \in \mathbb{R}^{r \times k}$ ，而秩 r 远小于原矩阵维度。这样一来，真正要训练的参数数量就从完整矩阵降到了两个小矩阵。

这背后的工程直觉很重要：很多下游任务不需要完全重写全部表示，只需要在某些方向上对原有权重做可控偏移。LoRA 假设这种偏移可以被低秩结构很好近似，于是把训练成本降了下来。

5.2 LoRA 的价值不只是省显存，还在于可插拔

Slides 专门展示了“swap matrices = swap tasks”的画法，意思是：底层基础模型不变，只需替换不同任务对应的 LoRA adapter，就能让同一个基座承担不同功能。这种可插拔能力非常适合多任务部署，也让分享和分发模型变得更轻量。

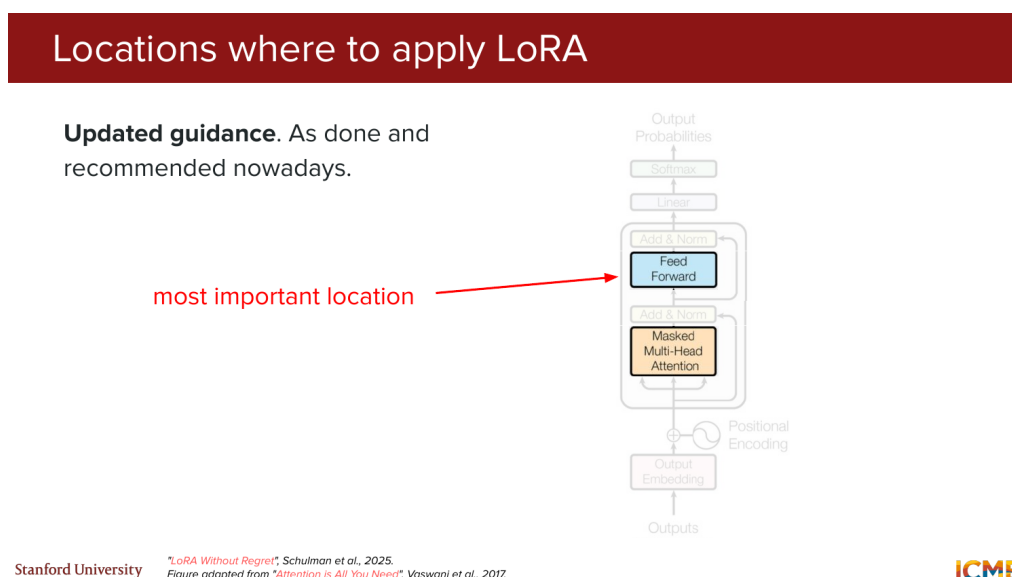


图 4: 课程给出的更新版建议强调：LoRA 通常最值得优先放在前馈层等高影响位置，而不是无差别地加到所有线性层。

5.3 LoRA 应该加在哪里，以及它的训练动态有什么特殊性

课程对 LoRA 的讲解没有停留在定义层面，而是进一步讨论“放在哪里更有效”。Slides 对比了原始论文中的做法与更新后的经验法则，指出如今更常见也更推荐的选择，是优先覆盖最关键、对表示能力影响最大的层，尤其是前馈网络中的某些权重。

老师还点出两条容易被忽略的训练动态：

- LoRA 往往需要比 full fine-tuning 更高的学习率；
- 在很大的 batch 下，LoRA 的表现可能不如 full fine-tuning 稳定。

这提醒我们，LoRA 不是“无脑替代品”，而是一个带有自身超参数规律和优化习惯的训练方法。

5.4 QLoRA: 把冻结权重量化，再保留全精度 LoRA 分支

当显存继续成为主要瓶颈时，课程引入了 QLoRA。它延续 LoRA 的结构，但进一步把冻结的底座权重做量化存储，只把 LoRA 的小矩阵保留在全精度里参与训练，于是同时获得两种好处：底座权重占用更少显存，adapter 训练仍然保持较好数值表现。

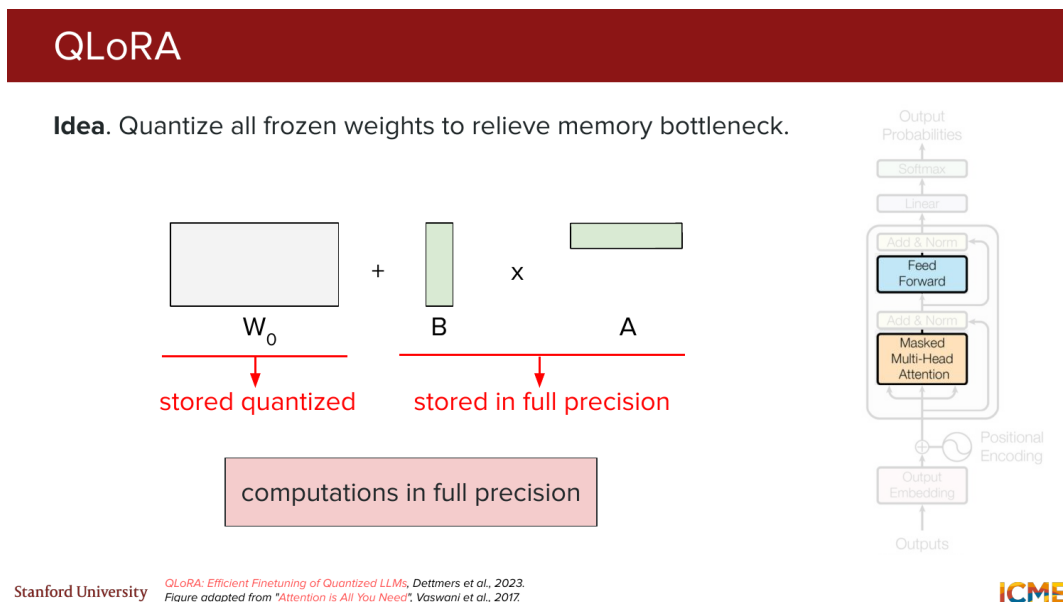


图 5: QLoRA 的结构示意：底座权重以量化形式存储，LoRA 分支保留全精度，前向计算仍在高精度路径中完成。

课程还解释了两个关键技巧。第一个是 **NF4**，即 4-bit NormalFloat。它不是简单做均匀切分，而是依据近似正态分布的分位点来设置量化区间，更贴近神经网络权重的统计形状。第二个是 **double quantization**，即连量化常数本身也继续压缩，进一步减少额外开销。

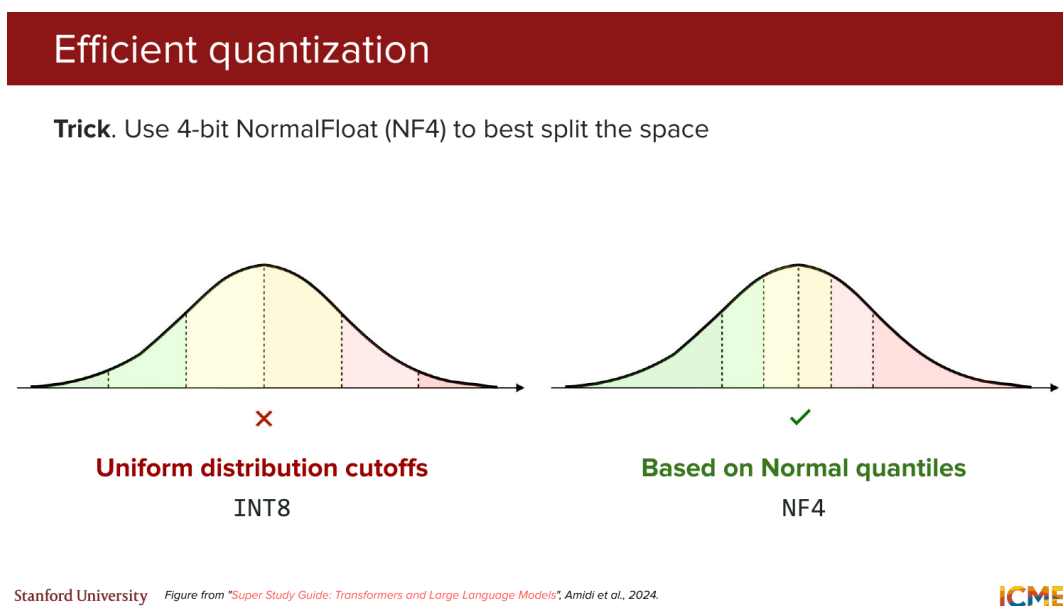


图 6: NF4 的直觉是按近似正态分布的分位点切分数值范围，而不是均匀切分，从而更有效利用 4 bit 表示能力。

5.5 QLoRA 说明了“显存预算”本身就是建模约束

Slides 给出了数量级上的收益：在 LLaMA 65B 一类模型上，QLoRA 能把微调显存需求降到一个更可操作的水平，同时 double quantization 还能再带来额外节省。课程希望学生记住的不是某个精确百分比，而是更一般的原则：一旦显存预算成为硬约束，模型适配方法本身就必须围绕显存来设计。

LoRA 与 QLoRA 的关系

LoRA 解决的是“只训练少量增量参数”，QLoRA 解决的是“连冻结底座也要尽量省显存”。前者强调参数高效，后者强调参数高效加存储高效。

5.6 本章小结

LoRA 把完整微调改写成低秩增量学习，适合在共享基座上快速适配多个任务；QLoRA 则进一步量化冻结权重，让大模型在更小硬件预算上也能完成高质量微调。两者共同说明，下游适配已经不只是算法问题，更是资源约束下的系统设计问题。

6 总结与延伸

Lecture 4 串起了 LLM 生命周期中的第一批关键环节。首先，训练范式从“单任务单模型”转向“预训练 + 调优”的两阶段结构，解释了为什么一个基础模型可以服务多种任务。其次，预训练之所以昂贵，不只是因为模型大，更因为它要在庞大异质数据上长期优化 next-token objective，同时面对规模律、数据混合、知识截止与成本治理等现实问题。再次，真正把大模型训起来，必须处理显存瓶颈、分布式并行、FlashAttention 与混合精度等系统层问题。最后，SFT、LoRA 与 QLoRA 则把注意力从“如何学到通用能力”转向“如何以更低成本把能力塑造成产品可用的行为”。

从课程安排上看，这一讲实际上是在为后两讲铺路。SFT 会自然过渡到 preference tuning，LoRA 和 QLoRA 也为后来讲到的 RLHF、DPO、reasoning model training 提供了现实可行的训练工具。如果把 Lecture 4 压缩成一句话，那就是：**现代 LLM 训练既是统计学习问题，也是显存、带宽、并行和部署预算共同约束下的系统工程。**